

# Mini Jogo de Damas com Inteligência Artificial

Aluno: João Victor Borges Carvalho

Repositório no github: <https://github.com/Jkvzin/damas-com-ia.git>

Este projeto é uma implementação em Java de um jogo de Damas (tabuleiro 6x6) com uma Inteligência Artificial robusta baseada no algoritmo **Minimax**, aprimorada com a **Poda Alpha-Beta** e **Simulações de Monte Carlo**.

O sistema foi desenhado de acordo com regras acadêmicas específicas decididas em aula, garantindo um jogo justo, restritivo e com uma IA escalável capaz de jogar do nível mais básico até uma precisão invencível.

## Regras e Movimentação das Peças

O jogo segue uma adaptação das regras clássicas de Damas para um tabuleiro reduzido de 6x6. As regras de movimentação foram construídas da seguinte maneira:

### Peças Comuns

- **Movimento Simples:** As peças comuns movem-se apenas uma casa na diagonal para a frente.
- **Proibição de Recuo:** Peças comuns não podem se mover ou realizar capturas para trás. O recuo é estritamente proibido.

### Damas (A Lógica Especial)

Quando uma peça atinge a última linha do tabuleiro (lado do adversário), ela é promovida a **Dama**.

- **Liberdade Direcional:** A Dama ganha a habilidade de se mover e capturar em todas as quatro direções diagonais (para frente e para trás).
- **Distância de Voo:** Ao contrário de algumas variantes simplificadas, a Dama possui movimento longo. Ela pode deslizar por várias casas vazias na diagonal em um único turno.
- **Captura à Distância:** A Dama pode capturar uma peça inimiga que esteja a qualquer distância na mesma diagonal, desde que o caminho até ela esteja livre e que a casa imediatamente após a peça inimiga também esteja vazia.

## A Lógica de Captura

A captura ocorre quando uma peça salta por cima de uma peça adversária adjacente (ou à distância, no caso da Dama), aterrissando na casa vazia seguinte. O projeto implementa com rigor a **Lei da Maioria Absoluta**:

- **Obrigatoriedade:** Se existe uma oportunidade de captura, o jogador (humano ou IA) é **obrigado** a capturar.
- **Maioria:** Se houver múltiplos caminhos de captura possíveis, o sistema filtra e bloqueia os menores. O jogador é forçado a escolher a rota que elimine o **maior número de peças inimigas** no mesmo turno (combo).
- **Múltiplos Saltos (Combos):** Se, após uma captura, a peça que saltou puder capturar novamente, o turno continua e a peça deve continuar saltando até que não existam mais alvos.

A interface gráfica foi desenhada para impedir ativamente jogadas ilegais, não permitindo cliques em peças ou casas que não respeitem a Lei da Maioria.

## A Inteligência Artificial

A "mente" por trás do jogo é movida pela classe `Árvore.java`, que constrói matematicamente todas as realidades alternativas do tabuleiro para escolher o melhor lance.

### O Algoritmo Minimax e a Árvore de Possibilidades

Para a IA decidir uma jogada, ela gera uma árvore de possibilidades:

1. A **raiz** da árvore é o tabuleiro no estado atual.
2. A **camada 1** são todas as jogadas que a IA pode fazer agora.
3. A **camada 2** são todas as respostas possíveis que o jogador humano poderia dar a cada uma dessas jogadas, e assim por diante.

O algoritmo **Minimax** assume que tanto a IA quanto o Humano jogarão perfeitamente. A IA sempre busca maximizar o seu próprio placar (Max), enquanto assume que o humano sempre buscará minimizar o placar da IA (Min). No fundo da árvore, apliquei uma função heurística para avaliar o tabuleiro (dar nota). A nota sobe de volta para a raiz, e a IA escolhe a jogada que lhe garante o cenário menos pior, mesmo se o humano jogar perfeitamente.

### A Função de Avaliação (Heurística)

Como o Minimax não pode prever o final da partida em todas as jogadas, ele precisa de uma "nota" matemática para avaliar se um tabuleiro é bom ou ruim no momento em que para de calcular. A função heurística desenhada soma e subtrai as peças e

o posicionamento de ambos os jogadores, chegando a um **Score** final (Pontuação IA - Pontuação Humano).

Os pesos matemáticos definidos para a Heurística são:

- **Vitória Matemática (Inimigo sem peças/travado):** +100000 pontos (Prioridade Absoluta).
- **Dama:** 300 pontos. (Uma dama vale por 3 peças comuns, incentivando fortemente a promoção).
- **Peça Comum:** 100 pontos.
- **Inteligência Posicional (Bônus Geográfico):** A IA recebe uma expansão de consciência geográfica em todas as dificuldades. Peças ganham pequenos bônus somados aos seus 100 pontos base dependendo de onde estão pisando:
  - **Domínio do Centro:** +20 pontos (Casas do centro garantem o controle das rotas e mais mobilidade).
  - **Bordas Laterais:** +10 pontos (Bordas servem de escudo, pois peças ali não podem ser capturadas).
  - **Avanço Ofensivo:** +5 pontos por linha avançada no território inimigo (Incentiva a IA a jogar ativamente para frente em vez de "camperar" na defesa).

## A Poda Alpha-Beta (Alpha-Beta Pruning)

Uma árvore Minimax cresce de forma exponencial (crescimento fatorial). Em poucas rodadas simuladas, existem milhões de tabuleiros gerados, o que congelaria qualquer computador. Para resolver isso, implementei a **Poda Alpha-Beta**.

**O que é e como funciona:** A Poda Alpha-Beta é uma técnica de otimização matemática acoplada ao Minimax. Ela mantém um registro de duas variáveis enquanto desce na árvore:

- **Alpha:** O melhor valor garantido para a IA até o momento.
- **Beta:** O melhor valor garantido para o Humano até o momento.

Enquanto o algoritmo está avaliando as opções, se ele descobre que um determinado ramo da árvore levará a uma situação que é **pior** do que uma alternativa que ele já encontrou antes, ele **poda** todo esse ramo e nem perde tempo calculando seus filhos.

*Exemplo prático:* Se a IA já achou uma jogada que garante capturar uma peça, e ela começa a analisar uma segunda jogada e vê que o humano pode responder com uma captura múltipla, ela imediatamente para de calcular o resto das possibilidades daquela segunda jogada, porque ela **sabe** que nunca vai escolhê-la. Isso permite

que a minha IA calcule 12 níveis de profundidade em milissegundos, analisando centenas de milhares de nós com a máxima eficiência computacional.

### **Dificuldades e Monte Carlo (Níveis 1 a 9)**

O nível de inteligência da IA não é fixo. Implementei uma escala progressiva de 1 a 9 que mistura Minimax com simulações probabilísticas (**Simulação de Monte Carlo**).

A simulação de Monte Carlo é uma técnica matemática que utiliza amostragem aleatória massiva para modelar incertezas e prever resultados de eventos complexos.

- O número da dificuldade **X** (de 1 a 9) dita a profundidade real calculada matematicamente pela árvore Minimax.
- Assim que o Minimax atinge a camada **X**, ele não avalia o tabuleiro imediatamente. Em vez disso, ele entra em um loop onde realiza jogadas "aleatórias" (simulação em alta velocidade por ambos os lados) até completar **10 turnos simulados** no futuro.
- Ao chegar no 10º turno, ele finaliza a simulação e avalia o tabuleiro.

#### **O resultado prático:**

- No **Nível 1**, o Minimax pensa apenas 1 jogada à frente e preenche as outras 9 na base da sorte. A IA joga de forma caótica, amadora e altamente dependente do acaso.
- No **Nível 8**, o Minimax pensa 8 jogadas exatas de forma genial, e só conta com a sorte para simular as últimas 2 rodadas, sendo extremamente letal e intencional.

### **A Dificuldade Máxima (Nível 10)**

Para jogadores que querem o desafio final, a Interface disponibiliza a "Dificuldade Máxima (10)". Neste modo, as simulações probabilísticas de Monte Carlo são completamente desativadas. O algoritmo entra no modo **Minimax Puro** acoplado à Poda Alpha-Beta com uma profundidade bruta de **12 nós** (testei e consigo ir além disso, até 16 nós rodou tranquilo). Ele varre matematicamente todas as jogadas e utiliza uma Heurística Posicional agressiva para dominar o centro do tabuleiro, promover damas o mais rápido possível e esmagar o oponente de forma impiedosa e sem erros.